

Team Number:	apmcm2309364
Problem Chosen:	A

2023 APMCM summary sheet

In order to cope with Apple picking robots have low recognition rate, slow speed and low accuracy in image recognition. we use the improved Hough circle transformation method to establish the first type of model for identifying apples. To better identify apples more accurately in complex situations, we extracted the feature value information of apples through CNN convolutional network, conducted deep learning on the provided apple images, and established a second type of apple recognition model based on YOLOv4 and VGG19. In addition, when calculating the quality of the apples in the image, we invented a brightness-estimation method ourselves to make the calculated apple quality more consistent with the actual situation.

Question 1 requires us to identify the number of apples in the image. We use Lab color space conversion and Gaussian filtering method to preprocess the image. Taking advantage of the characteristic that ripe apples are red, we drew the a^* channel map in the Lab color space and binarized it. Then, taking advantage that apples can be approximately regarded as circles in a two-dimensional plane, we used an improved Hough circle transformation method, and the number of apples identified in Appendix 1 was 3094.

Questions 2 and 3 require us to identify the position and maturity of apples in the image. We used the YOLOv4 network in the CNN convolutional network for deep learning, input the image data of artificial frame selection and recognition for model training, and used the trained The model was used for identification, and it was found that the results of the apple position were similar to the results of the first model, indicating that the model was well trained. When calculating the maturity in question 3, we also defined a maturity function H to quantify the maturity of the apples.

Question 4 requires us to calculate the mass of apples. We further considered the imaging principle of "large near, small far", analyzed the impact of focal length and the distance between the apple and the lens on the results, and believed that if only calculating the quality of the apple based solely on the area of the apple's two-dimensional image may be very different from the actual situation. Based on this, we proposed a brightness-estimation method to correct the results, and calculated the corrected apple quality using the data results obtained from the first three questions.

Question 5 requires us to identify apples from different fruits. We noticed that yolov4 is a target detection model, which is more biased towards image classification, and the number of samples in Appendix 2 is too large, and the efficiency of manually dividing the training set is low. Therefore, we used the VGG19 network with more convolutional layers to train the model. Compared with YOLOv4, it has higher efficiency and learning depth.

The innovation of this article is that we use geometric methods, deep learning methods and build models from different angles, and analyze and compare the results obtained, so that the model has high credibility. At the same time, we also considered many practical problems that may not have been considered in the questions, and gave us our solutions.

Keywords: Image recognition Hough circle transform, CNN convolutional network
deep learning YOLO network VGG19

Contents

1. Introduction	1
1.1 Question Background	1
1.2 Restatement of Problem	1
2. Problem analysis	2
2.1 Analysis of Problem One	2
2.2 Analysis of Problem Two	2
2.3 Analysis of Problem Three	2
2.4 Analysis of Problem Four	3
2.5 Analysis of Problem Five	3
3. Model Hypothesis	3
3.1 Assumption 1	3
3.2 Assumption 2	4
3.3 Assumption 3	4
3.4 Assumption 4	4
4. Notations	5
5. Model Establishment and Solution of Problem 1	6
5.1 Establishment of Apple Recognition Counting Model	6
5.1.1 <i>Image Data Preprocessing</i>	6
5.1.2 <i>Methods for Identifying Apples in Images</i>	9
5.1.3 <i>Statistical Methods for Counting Apples and Their Improvements</i>	11
5.2 Results and Histogram of apple distribution	14
6. Model Establishment and Solution of Problem 2	15
6.1 Establishment of the Apple Position Determination Model	15
6.1.1 <i>Principle Overview of YOLOv4 network</i>	15
6.1.2 <i>The Application of the YOLOv4 network and Results in this Problem</i>	16
6.1.3 <i>Comparison of Model Recognition and Sensitivity Analysis</i>	20
6.2 Results and 2D Scatter Plot of Geometric Coordinates	20
7. Model Establishment and Solution of Problem 3	21
7.1 Establishment of apple maturity detection model	21
7.1.1 <i>The Application of YOLOv4 Network and Results in this Problem</i>	21
7.1.2 <i>Establishment of numerical calculation model for maturity</i>	22
7.2 Results and Histogram of apple ripeness distribution	22
8. Model Establishment and Solution of Problem 4	24
8.1 Preliminary estimates for Apple	24
8.2 Radance and radiation intensity	25
8.3 Establishment and calculation optimization of ranging model	26
8.4 Estimate of actual fruit size and mass	29
8.5 Results and mass distribution histogram	30

9. Model Establishment and Solution of Problem 5	31
9.1 Establishment of an improved apple recognition model	31
9.1.1 <i>Principle Overview of VGG19 network</i>	31
9.1.2 <i>The Application of the VGG19 network</i>	33
9.2 Results and histogram for all Apple image ID numbers	34
10. Model Evaluation and Further Discussion	35
10.1 Merit	35
10.2 Shortcoming	35
10.3 Improvement Measures	36
11. References	36
12. Appendix	38

I. Introduction

1.1 Question Background

Apple-picking robots have become a recent hotspot. In the past, apple harvesting relied mainly on manual labor, requiring a large workforce for several days during the apple's ripening period. However, with the aging rural workforce and the trend of young people leaving villages for work, many places are experiencing a shortage of labor during the apple harvesting season. To address this issue, people have started developing apple-picking robots. But due to variations in orchard environments and controlled experiments, the widespread adoption and application of various apple-picking robots globally have not been ideal.

At the same time, existing robots mostly cannot achieve precise identification of obstacles such as leaves, branches, and fruits. Direct harvesting may potentially damage the fruits or even the harvesting equipment, impacting efficiency and fruit quality. Additionally, there are shortcomings in the recognition and differentiation technologies for different harvested fruits, making post-harvest identification of apples challenging.

Therefore, it is essential to establish an efficient and accurate apple image recognition model. This paper analyzes features such as the number, position, ripeness, and quality of apples in apple images by using image segmentation and recognition techniques, mathematical models, and the deep learning network, and then proposes a feasible apple image recognition algorithm to address these challenges.

1.2 Restatement of Problem

Considering the background information and restricted conditions in the problem statement, the following problems should be solved:

Question 1: Preprocess the image dataset in Attachment 1, analyze the relationship between the number of apples and various parameters in the image dataset, determine the required image features, establish a mathematical model, and count the number of apples in each image. Then draw a histogram of the distribution of the number of apples to visualize the data.

Question 2: In the previous question, we have already determined the number of apples. Next, we need to identify the position of each apple in the image. We establish a Cartesian coordinate system with the lower-left corner of the image as the origin. Since one image may contain multiple apples, we need to assign a unique number to each apple and store the coordinate information for all apples. Then, we will draw a scatter plot of the apple coordinates to visualize the data.

Question 3: In this question, we need to find a reasonable method to calculate the ripeness of apples and then apply this method to perform the calculations. The results should be visualized by creating a histogram that represents the distribution of ripeness for all apples, to visualize the data.

Question 4: In question 1 and 2, we have already determined the number and positions of apples. Now, we need to calculate the two-dimensional area of the apples based on this information. The key lies in determining the boundaries of the apples. Additionally, we are required to estimate the mass of apples. Since we cannot directly measure the mass, we can only obtain an estimate. Therefore, we need to establish the relationship between the two-dimensional area of apples in the

image and their mass for estimation. Subsequently, we will visualize the distribution of the mass of all apples by creating a histogram.

Question 5: In this question, the task is to identify apples from various fruit images. This involves a more in-depth extraction of apple-specific image features, training a model for apple recognition, and applying it to determine which images in Attachment 3 contain apples. Subsequently, the distribution of their ID numbers will be visualized through the creation of a histogram for data analysis.

II. Problem analysis

2.1 Analysis of Problem One

In Problem 1, based on the provided apple image data in Attachment 1, we aim to identify features that can help us recognize relevant information about apples. It is observed that ripe apples exhibit the characteristics of being red and having an approximately spherical appearance (which can be considered circular in a two-dimensional image). Based on these two features, we can identify apples in the image. Before establishing the model, we need to preprocess the image data.

Here, we convert the original RGB image to the Lab image. Subsequently, we filter the a^* component (representing the spectrum change from green to red) to obtain the A-channel image in the Lab color space. We perform binarization on this A-channel image and apply Gaussian filtering for noise reduction. Next, utilizing the circular features, we employ an improved Hough circle transform to detect and count the number of circles in the image. Additionally, we merge circles with high overlap. Finally, we solve the data in Attachment 1 and plot the histogram.

2.2 Analysis of Problem Two

In Problem 2, we need to identify the positions of apples in each image. Building on the preprocessing of all images in problem one and obtaining the number of apples in each image, we can roughly determine the positions of apples using the model developed in Problem 1 with the obtained centroids after numbering them. However, after analysis and comparison, we found that there is still a certain deviation between the geometric center of the apple and the centroid obtained from the Hough circle.

Therefore, we adopted a new recognition method based on deep learning to determine the coordinates of each apple, using the YOLOv4 network. We first train the model using images with clear and distinctive features, then apply the trained model to all images in Attachment 1. Finally, we compare the recognition results with the training results and the results from Problem 1, conduct sensitivity analysis on the model, and plot a scatter plot of the geometric coordinates of all apples.

2.3 Analysis of Problem Three

In Problem 3, we can follow the approach used in Problem 2, utilizing the YOLOv4 network for model training. However, during training, it is necessary to annotate the manually added bounding boxes with the ripeness status of the apples (ripe, semi-ripe, and unripe). Subsequently, the model is trained, and the trained model is applied to identify the ripeness status of each apple in the

images. We obtain information about the ripeness status and confidence level of each recognized apple. Based on this information, we can design a function to measure apple ripeness, allowing us to calculate the ripeness of each apple. Finally, a histogram depicting the ripeness distribution of all apples is generated for further analysis.

2.4 Analysis of Problem Four

In Problem 4, we want to solve in this question is to calculate the area of a two-dimensional image and estimate the mass of the apple. For the area problem, we might as well use the radius rad we obtained in the first question to calculate it as a simple estimate. As we said in the hypothesis, we assume that the density of apples does not change significantly during the development cycle, which is a fixed value, which may be recorded as ρ . In addition, our team noticed that there are obvious differences in the image in Appendix 1 The problem of large near and far small results in the error result that for some apples in the image, the actual apples are larger and the apples in the image are smaller. To this end, our team reviewed relevant literature on calculating the actual distance of objects through two-dimensional plane images. Unfortunately, the methods reviewed in the literature were not applicable to the dilemma faced by our team. After discussion, we finally proposed a new method - the brightness-estimation method to approximately estimate the actual size of the apple. In this method we need to know the focal length of the camera when shooting, and finally we draw the mass distribution histogram of all apples.

2.5 Analysis of Problem Five

This question requires us to distinguish apples from other fruits in the image, which requires us to extract more information features about apples, instead of calculating the location, maturity, quality, etc. of apples like the previous questions. If we continue to use the YOLOv4 network, when the number of training times is high, the time complexity will be too high. Therefore, we should choose a deep learning algorithm with more convolutional layers and faster computing speed so that we can be more accurate. and efficiently identify apples. At the same time, considering that there are too many Apple ID numbers, we choose to cluster every 1,000 consecutive IDs, count the number of apple images within these 1,000 IDs, and then draw a histogram.

III. Model Hypothesis

3.1 Assumption 1

There are often small and numerous apples in the background of the image that are difficult to count. For the purpose of ensuring the speed and accuracy of image recognition, we will ignore the small apples in the image.

Explanation: This assumption is reasonable and sufficient. For the hypothetical situation, in real life, you can better identify the apples by getting closer to reduce the distance, taking partial images, and changing the shooting angle, instead of forcing it on the numerous and difficult-to-count apples in the background. detection, which cannot guarantee the speed and accuracy of recognition.

3.2 Assumption 2

The fruit volume and density of apples are basically unchanged during the ripening period of the fruit. If small errors are ignored, they can be regarded as fixed values.

Explanation: Generally speaking, the fruit develops during the fruiting period and the overall shape continues to expand. When the fruit shape is basically stable, it begins to enter the fruit maturity stage. The external manifestation of the fruit during the ripening stage is mainly the change in the color of the peel. Its shape and quality are basically unchanged, and its density can also be considered unchanged.

3.3 Assumption 3

All apples are red apples. Green apples are considered immature red apples, and we believe that their degree of ripeness is linearly related to the degree of the apple's gradient from green to red.

Explanation: The appearance of green apples and unripe red apples are very similar, and since the image data we can obtain are only two-dimensional images with lower pixels, we cannot accurately distinguish between the two. Considering that most of the image data types are red apples, we can think that all apple types are red apples, and the ripeness of apples can only be distinguished by color, so it is assumed that apple color changes and ripeness changes are linear. The relevant ones make it easier for us to build models, and the gap with the actual situation is very small.

3.4 Assumption 4

In the image data given, all photos were taken under natural light with uniform and soft light. The objects captured were all still life, and no ghosting was produced during the shooting. The focal length of the shooting equipment used is the same.

Explanation: The time for picking apples is generally during the day, so there is no need for another strong light source, and the strong light source will have a great impact on the imaging and affect the recognition results. We believe that the light is uniform and soft natural light, which is easy to eliminate interference factors. Under normal circumstances, the position of the apple does not change much due to the influence of the external environment. At least we can think that the apple is stationary while shooting, eliminating the interference of ghost images during imaging on the recognition results. And since the imaging equipment of a picking robot is the same, in practical applications we can think that the shooting focal length of all pictures is the same.

IV. Notations

Symbol	Description
γ	Parameter used to describe the overlap rate between two circles
μ	Weighting factors used to measure maturity
H	Function used to represent the ripeness of an apple
$\tilde{H}$	Parameter used to represent average maturity
W_{img}	The average brightness of a certain area on a two-dimensional grayscale image
$G_{m \times n}$	Represents the gray value matrix of pixels on a two-dimensional grayscale image of size $m \times n$
$ A $	Sum all elements of matrix A

V. Model Establishment and Solution of Problem 1

5.1 Establishment of Apple Recognition Counting Model

5.1.1 Image Data Preprocessing

At first, we need to preprocess the image data provided in Attachment 1. We observed that the provided image data is in RGB format. In order to filter out the red color of apples in the image, we plotted the R-channel image. However, we noticed that due to interference from other colors (such as the brown color of tree branches), the resulting image after binarization is not ideal, as shown in the following figure:



(a) The original image. (b) The R channel image in the RGB color space after binary processing.

Figure 1 Comparison of original image and RGB grayscale image

After reviewing relevant literature and conducting analysis, we have determined that selecting the A channel image in the Lab color space (hereafter referred to as the A channel image) is a preferable choice. This is because the a^* component in the A channel image represents the spectrum change from green to red, which can effectively adapt to backgrounds predominantly composed of green leaves. Additionally, it helps mitigate interference from other colors.

Below is the process of converting the original image's RGB parameters to Lab parameters. Since RGB parameters cannot be directly converted to Lab parameters, we opt for using XYZ parameters as an intermediate step, i.e., $RGB \rightarrow XYZ \rightarrow Lab$. The conversion formulas are as follows:

Part1 : (RGB to XYZ)

Assuming r , g , and b are the pixel values for the three channels, with a range of $[0, 255]$, the conversion formula is as follows:

$$\begin{cases} R = gamma(\frac{r}{255.0}) \\ G = gamma(\frac{g}{255.0}) \\ B = gamma(\frac{b}{255.0}) \end{cases} \quad (1)$$

$$\gamma(x) = \begin{cases} \left(\frac{x + 0.055}{1.055}\right)^{2.4} & (x > 0.04045) \\ \frac{x}{12.92} & (otherwise) \end{cases} \quad (2)$$

The gamma function is used for non-linear tone mapping in images with the aim of enhancing contrast. The expressions mentioned above represent commonly used gamma functions.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = M \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (3)$$

Where the transformation matrix M is given by

$$M = \begin{pmatrix} 0.4124 & 0.3576 & 0.1805 \\ 0.2126 & 0.7152 & 0.0722 \\ 0.0193 & 0.1192 & 0.9505 \end{pmatrix}$$

Part2 : (XYZ to Lab)

$$\begin{aligned} L^* &= 116f(Y/Y_n) - 16 \\ a^* &= 500[f(X/X_n) - f(Y/Y_n)] \\ b^* &= 200[f(Y/Y_n) - f(Z/Z_n)] \end{aligned} \quad (4)$$

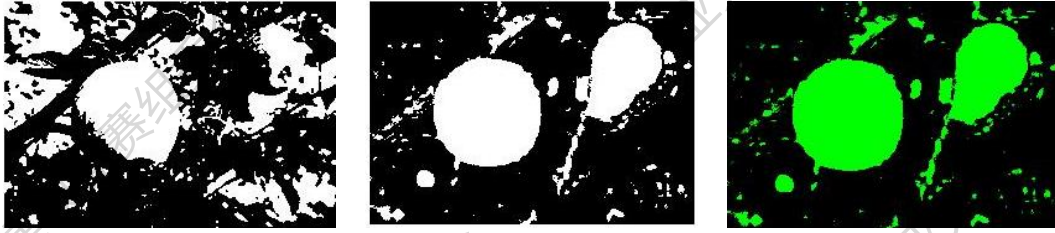
And

$$f(t) = \begin{cases} t^{\frac{1}{3}} & \text{if } t > \left(\frac{6}{29}\right)^3 \\ \frac{1}{3}\left(\frac{29}{6}\right)^2 t + \frac{4}{29} & \text{otherwise} \end{cases} \quad (5)$$

In the two formulas above, L^* , a^* , and b^* represent the values of the three channels in the final Lab color space. X , Y , Z are the values calculated after converting RGB to XYZ, and X_n , Y_n , Z_n are typically set to the default values of 95.047, 100.0, and 108.883.

Based on the above principles, we implemented the process of converting RGB to Lab through code. Then, in MATLAB, for the Lab image obtained, we filtered the a^* component and plotted its binarized image based on the L value. The result is the binary image of the A-channel in the Lab color space, as shown below:

Figure 2 The channel image



(a) The R channel image in the RGB color space after binary processing.

(b) The a* channel image in the Lab color space after binary processing.

(c) The a* channel image in the Lab color space.

After comparison, we can observe that the binarized image obtained using the Lab color space does indeed have better results than the one obtained using the RGB color space.

However, this alone is not sufficient. Through zooming in and contrasting the images, it is evident that there are numerous noise points in the image, and there is a noticeable aliasing effect at the edges, likely due to the low resolution of the image. This is unfavorable for the subsequent Hough Circle Transform. Therefore, we need to apply denoising techniques to make the image relatively smooth.

Here, we employ the method of Gaussian filtering:

Below is a brief introduction to the Gaussian filtering method:

Gaussian filtering is a process of weighted averaging for the entire image. The value of each pixel is obtained by taking the weighted average of its own value and the values of other pixels in its neighborhood. It is a form of linear smoothing filter commonly used to eliminate Gaussian noise. Gaussian filtering is widely applied in the denoising process of image processing.

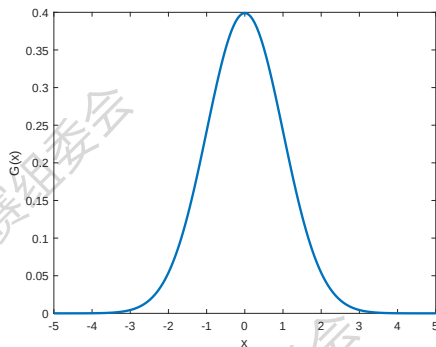
One – dimensional Gaussiandistribution :

$$G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

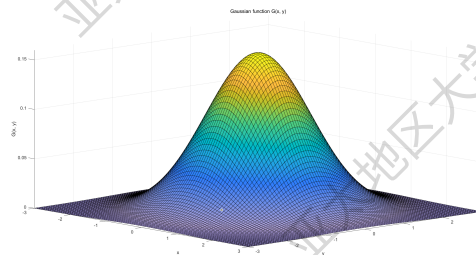
Two – dimensionalGaussiandistribution :

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Where σ is typically chosen as 1. The following is a schematic diagram of the Gaussian distribution.



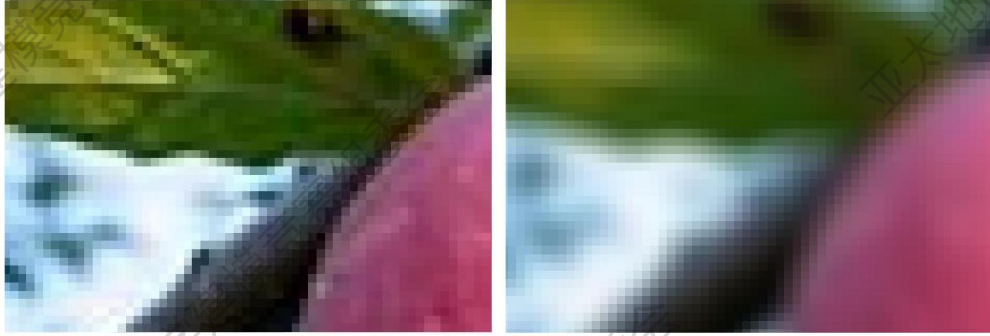
(d) Gaussian function G(x)



(e) Gaussian function G(x,y)

We adopt a two-dimensional Gaussian filtering here.

After debugging, we chose a suitable Gaussian kernel to apply Gaussian filtering to the previously obtained a* channel image. The resulting denoised image, when compared to the original, appears significantly smoother.



(f) The image before filtering.

(g) The image after filtering.

Figure 3 Comparison chart before and after Gaussian filtering

At this point, we have completed the preprocessing of the image data.

5.1.2 Methods for Identifying Apples in Images

After completing the preprocessing of the image data, our next step is to devise a method to identify apples in the processed images and determine the boundaries of the apples, facilitating subsequent counting and statistics.

Considering the characteristic nearly circular shape of apples in two-dimensional images, we adopted an improved Hough circle transform method to identify apples in the image. The following is a brief description of the Hough circle transform:

The Hough circle transform is an extension of the classical Hough transform, which is used for detecting lines in images. The principle is as follows:

1. We can use (a, b, r) to define a circle with center (a, b) and radius r .
2. If a circle passes through the point (x_1, y_1) , then it satisfies the equation:

$$(x_1 - a)^2 + (y_1 - b)^2 = r^2$$

3. Therefore, all circles passing through the point (x_1, y_1) can be represented as $(a_1(i), b_1(i), r_1(i))$, where $r_1 \in (0, +\infty)$. Each value of i corresponds to a different circle, and $(a_1(i), b_1(i), r_1(i))$ represents an infinite set of circles passing through the point (x_1, y_1) .
4. All circles passing through the point (x_1, y_1) can be represented as $(a_1(i), b_1(i), r_1(i))$, all circles passing through the point (x_2, y_2) can be represented as $(a_2(i), b_2(i), r_2(i))$, and all circles passing through the point (x_3, y_3) can be represented as $(a_3(i), b_3(i), r_3(i))$. If these three points lie on the same circle, then there exists a set of values (a_0, b_0, c_0) such that $a_0 = a_1(k) = a_2(k) = a_3(k)$, $b_0 = b_1(k) = b_2(k) = b_3(k)$, and $r_0 = r_1(k) = r_2(k) = r_3(k)$, meaning that these three points simultaneously lie on the circle (a_0, b_0, c_0) .

5. Analyzing all circles passing through the point (x_1, y_1) , when $r_1(i)$ is determined, the trajectory of $(a_1(i), b_1(i))$ is a circle with center $(x_1, y_1, r_1(i))$ and radius $r_1(i)$. Therefore, all circles $(a_1(i), b_1(i), r_1(i))$ form a cone with vertex at $(x_1, y_1, 0)$ and a cone angle of 90 degrees. The intersection point A of the three cones is the circle passing through these three points simultaneously. The point A with the highest cumulative count is a potential center of the circle.

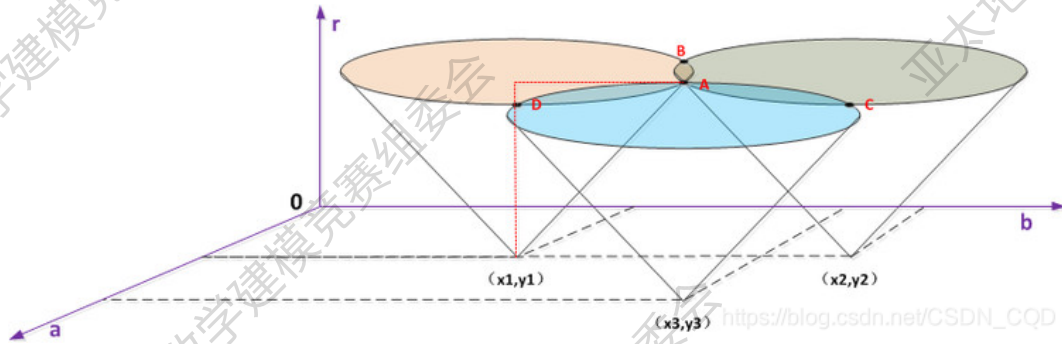


Figure 4 Hough Circle Transform diagram (Image Sourced from the internet).

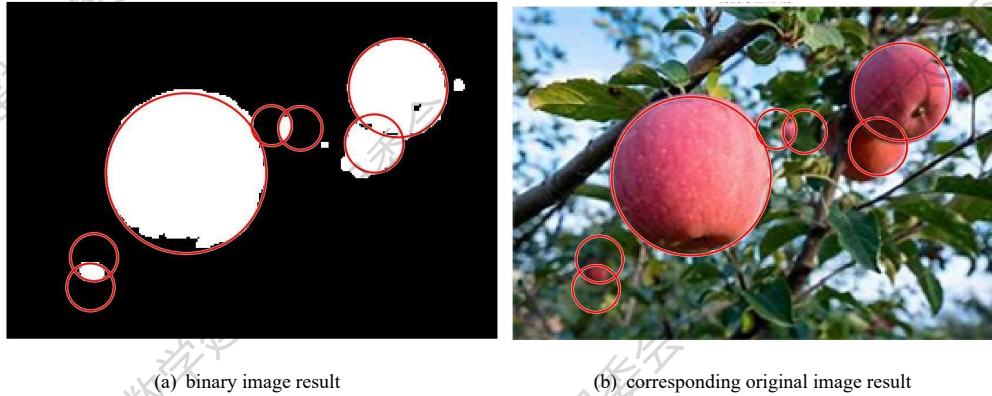
The Hough circle transform can accurately identify some circles that may exist in the image. However, this method involves a massive computational load, resulting in slow code execution. Therefore, we opted for a more efficient and flexible Hough gradient method. This method significantly reduces the computational load compared to the standard Hough circle transform and produces good results.

The following is an explanation of the implementation of the Hough gradient method for circle detection:

- **Step1:** Apply Canny edge detection to the obtained binary image to identify the positions of edge points in the image.
- **Step2:** For each non-zero point in the edge image, compute its local gradient by using the *imgradientxy()* function to calculate the first-order derivatives in the x and y directions, obtaining the gradient.
- **Step3:** Based on the gradients of the obtained edge points, accumulate each point in the accumulator array along the line specified by the slope (where the slope varies from the minimum to the maximum value as defined).
- **Step4:** Mark the position of each non-zero pixel in the edge image when simultaneously with each accumulation.
- **Step5:** Then, select candidate centers from these points in the two-dimensional accumulator, ensuring that these centers are both greater than a given threshold and larger than all their neighbors. These candidate centers are arranged in descending order based on their accumulation values, facilitating the appearance of the center with the most support pixels first.
- **Step6:** These pixels are then sorted based on their distances from the center. Starting from the minimum distance to the maximum radius, select the most suitable radius that encompasses all non-zero pixels.
- **Step7:** If a center receives sufficient support from non-zero pixels in the edge image and has an adequate distance from the previously selected centers, then it will be retained.

The points that are ultimately retained are the centers of the apple circles found by the Hough Gradient method. Based on the number of centers in the image, we can preliminarily estimate the approximate number of apples in the image.

Below are the results of the apple count obtained using this method.



This is the flowchart of our entire apple recognition and counting algorithm:

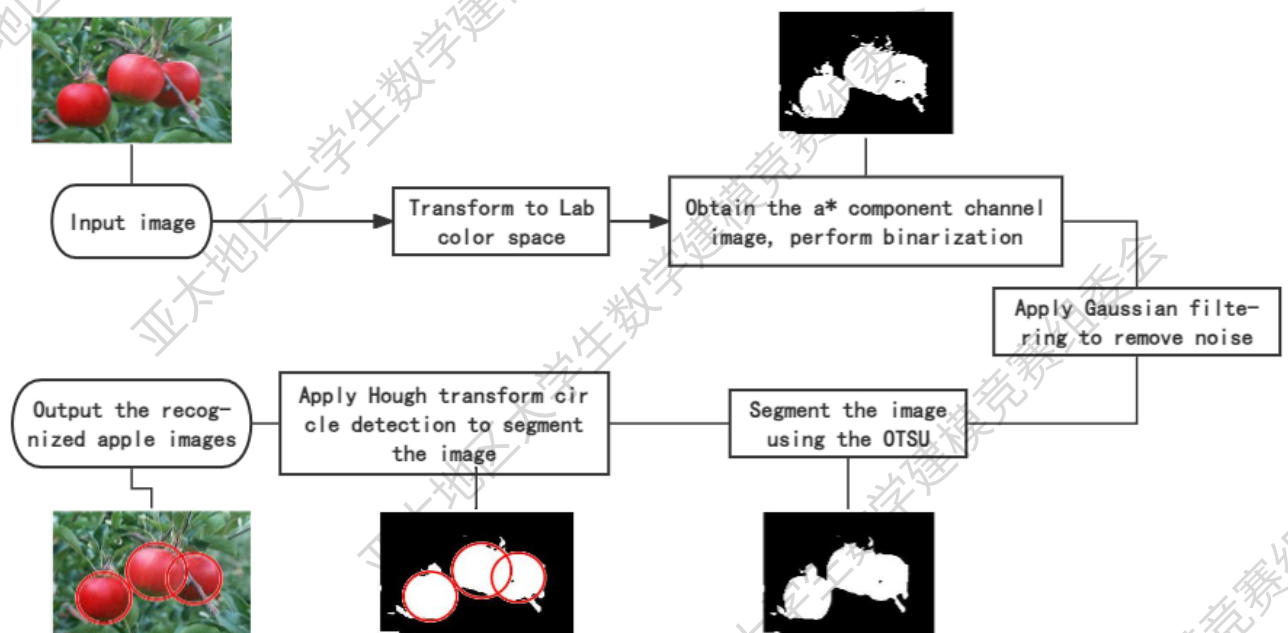


Figure 5 Flowchart of the Apple Recognition Algorithm.

5.1.3 Statistical Methods for Counting Apples and Their Improvements

In the previous section, we found a method to identify apples in the image. However, upon practical comparison, we discovered that due to factors such as lighting and shooting angles, the obtained count of apples may differ from the actual count, leading to significant discrepancies in the results.

Through analysis, we have identified that apples are not perfectly circular but appear as a combination of multiple circles. Therefore, the algorithm is prone to situations where one apple

is recognized as multiple apples, especially at the boundaries. To address this issue, we propose an optimization to the original model based on the overlapping area ratio of apple circles.

We propose the following improved method to address this issue:

1. Based on the previously obtained data for all circle centers (x_i, y_i) and their radii r_i we calculate the distances $d_{ij} (i \neq j) \in (0, r_i + r_j)$.
2. For ease of discussion, let's denote $R_1 = \max(r_i, r_j)$, $R_2 = \min(r_i, r_j)$, $d = d_{ij}$.
3. Next, we introduce the parameter $\gamma = \frac{R_1 - d}{R_2} \in (0, 1)$. The introduction of γ is intended to assist us in determining whether one of the circles should be eliminated in cases where two circles overlap.
4. For this purpose, we need to find a threshold value λ . When $\gamma > \lambda$, we consider that the overlap degree of the two circles is relatively high, and we remove the circle with the smaller radius N.
5. The number of circles retained is the optimized count of apples. Use the Count function to perform the counting.

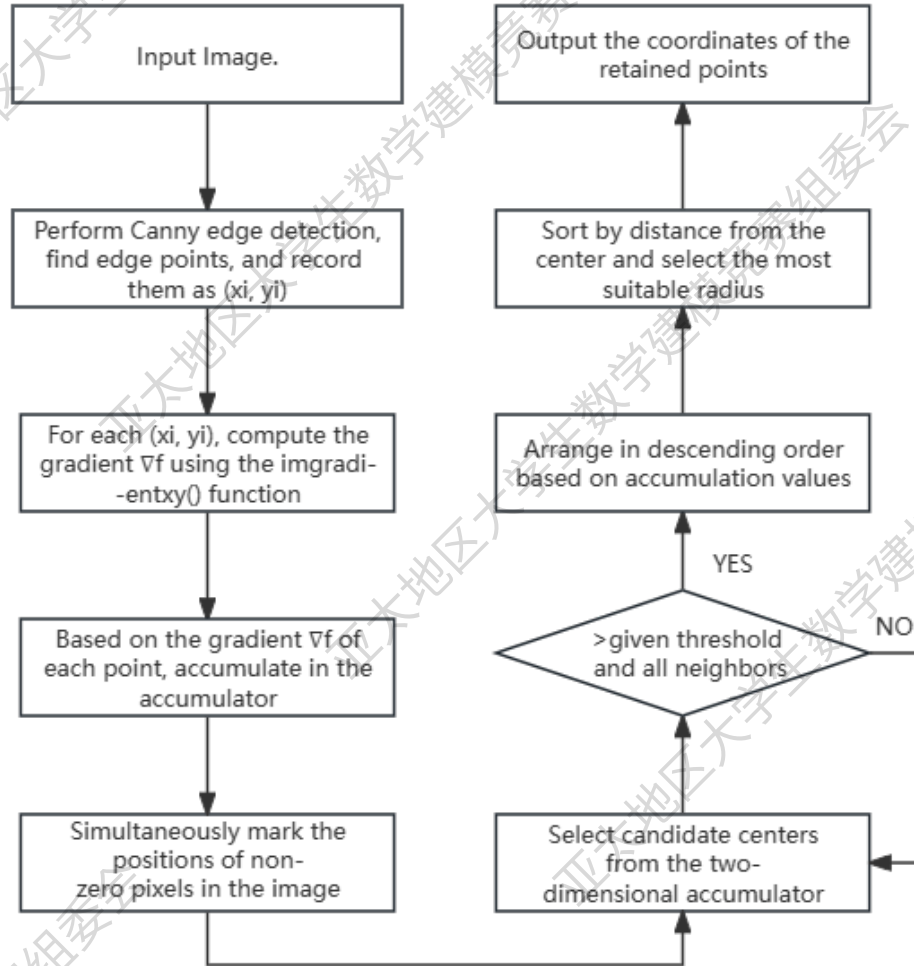


Figure 6 Flowchart of the Optimization Model for Eliminating Duplicate Circles.

Next, let's search for λ

When comparing the recognition image with the original image, we observe that when $\gamma < 0$ i.e.,

when $R_1 < d$, neither circle's center is inside the other circle. In this case, either there is no overlap or the overlapping area is very small. We can consider these as two separate apples without further discussion. When $\gamma > 1$, the smaller circle is completely encompassed by the larger circle, and we can consider it as one apple.

So, the λ we are seeking should be a value greater than 0 and less than 1.

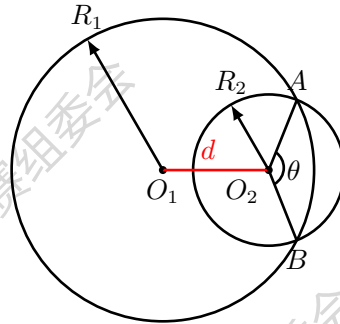


Figure 7 Schematic diagram of two circles overlapping

When $\gamma \in (0, 1)$, we can observe that there are two intersection points between the two circles. By connecting the center of the smaller circle with these two intersection points, we can obtain an angle θ . We can find that when R_1 and R_2 are both known, the area of overlap is related to θ and is negatively correlated (i.e., the smaller θ , the larger the overlapping area). From the images, we can

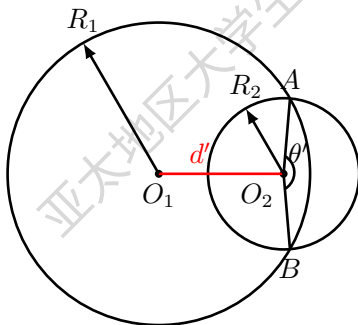


Figure 8 The bigger

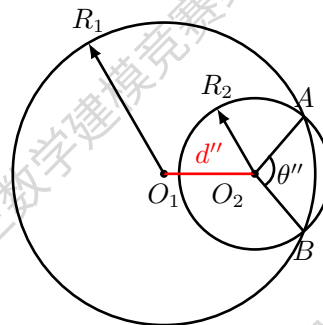


Figure 9 The smaller

also observe that as the value of γ increases, the distance d between them decreases, and θ continues to decrease as well.

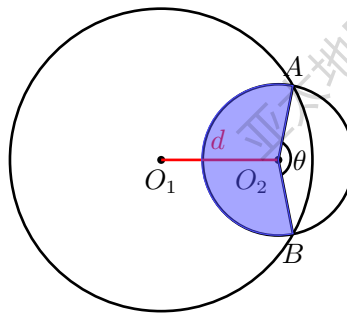


Figure 10 Overlap area i estimation diagram

From this, we can infer that the parameter γ has a positive correlation with the overlapping area.

In this case, we choose $\lambda=0.5$ as the threshold for determining whether two circles overlap. When $\gamma=0.5$, the angle θ is 120° . Through calculation, we find that at this point, the overlap degree of the two circles is above 70%. So we consider the overlap to be relatively high, and the two circles are identified as one apple.

5.2 Results and Histogram of apple distribution

The comparison between the results before and after optimization is shown below:

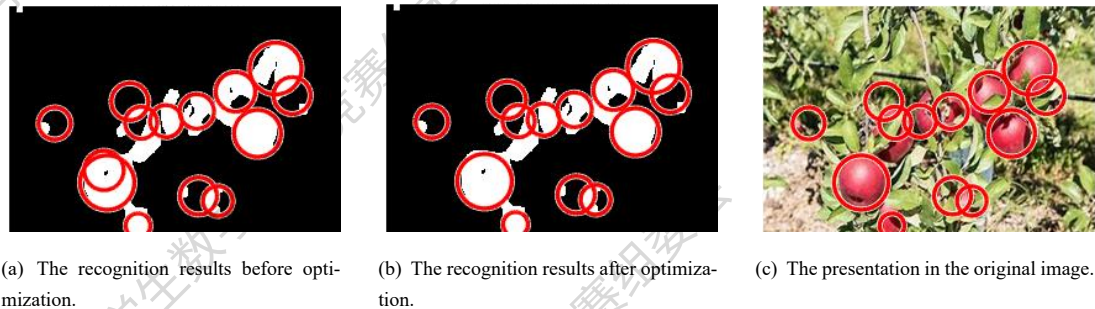


Figure 11 The comparison of effects before and after correction

From this, it is evident that the improved model does enhance recognition accuracy, and the recognized results align well with the distribution and count of apples in the original image. This can be used to calculate the quantity of apples.

According to this model, we will plot the obtained results using a bar chart, as shown below:

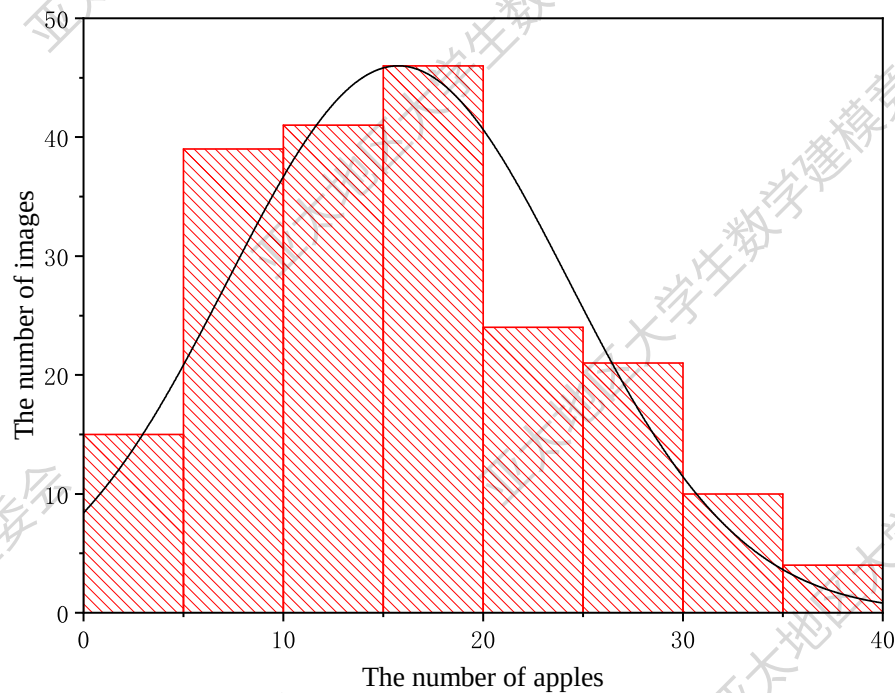


Figure 12 The histogram of the distribution of all apples in Attachment 1

On the basis of the histogram, we plotted a fitting curve and observed that the curve exhibits an approximate normal distribution. Since the provided images of apples are random and without a specific pattern, we can consider the obtained results to be satisfactory, indicating a high stability of the model.

VI. Model Establishment and Solution of Problem 2

6.1 Establishment of the Apple Position Determination Model

From the solution to the Problem 1, we observed that using the Hough Circle Transform method allowed us to determine the center of each Hough circle. However, there might be a certain deviation between the circle center positions and the actual center position of the apple cluster. To improve the model's accuracy, we adopted the YOLOv4 network and employed a deep learning approach to determine the coordinates of the apples. The results can be compared with those from the Problem 1 to assess the accuracy and rationality of the model. Below is an introduction to the YOLOv4 network.

¹

6.1.1 Principle Overview of YOLOv4 network

The YOLOv4 object detection network is a one-stage object detection network, consisting of three main parts: Backbone, Neck, and Head.

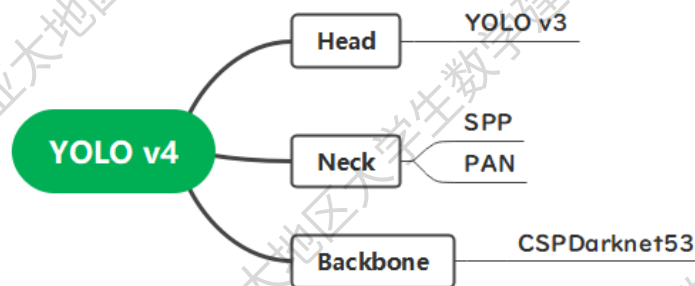


Figure 13 The composition of YOLOv4.

Through the **Figure 16**, we can clearly see its process. Below are detailed explanations:

- The YOLO v4 network uses CSPDarkNet-53 as the backbone for extracting features from the input images. The backbone has five residual block modules, and the feature map outputs from the residual block modules are fused at the neck of the YOLO v4 network.
- The SPP module in the neck concatenates the max-pooling outputs of the low-resolution feature map to extract the most representative features. The SPP module uses kernels of size 1-by-1, 5-by-5, 9-by-9, and 13-by-13 for the max-pooling operation. The stride value is set to 1.

¹When using the YOLOv4 algorithm, we referred to the references in MATLAB Help, the link is as follows
<https://ww2.mathworks.cn/help/vision/ug/object-detection-using-yolov4-deep-learning.html>

Concatenating the feature maps increases the receptive field of backbone features and increases the accuracy of the network for detecting small objects. The concatenated feature maps from the SPP module are fused with the high-resolution feature maps by using a PAN. The PAN uses upsampling and downsampling operations to set bottom-up and top-down paths for combining the low-level and high-level features.

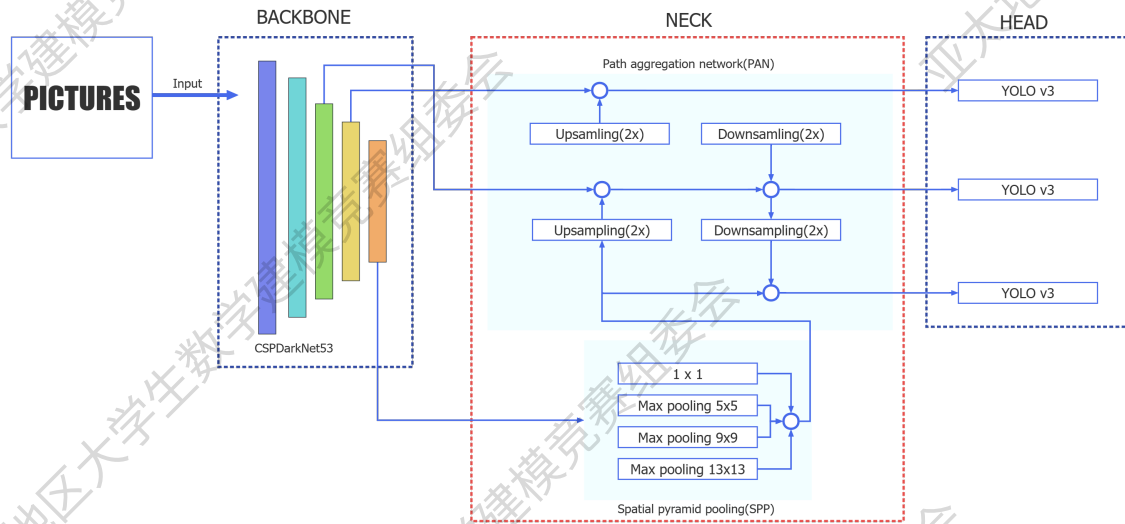


Figure 14 YOLOv4 Network Algorithm Flowchart.

- The PAN module outputs a set of aggregated feature maps to use for predictions. The YOLO v4 network has three detection heads. Each detection head is a YOLO v3 network that computes the final predictions. The YOLO v4 network outputs feature maps of sizes 19-by-19, 38-by-38, and 76-by-76 to predict the bounding boxes, classification scores, and objectness scores.
- Tiny YOLO v4 network is a lightweight version of the YOLO v4 network with fewer network layers. The tiny YOLO v4 network uses a feature pyramid network as the neck and has two YOLO v3 detection heads. The network outputs feature maps of size 13-by-13 and 26-by-26 for computing predictions.

6.1.2 The Application of the YOLOv4 network and Results in this Problem

Before training the model with the YOLOv4 network, we need to select samples from the provided apple image data in Attachment 1 and preprocess these data. The process of selecting samples and preprocessing is outlined below:

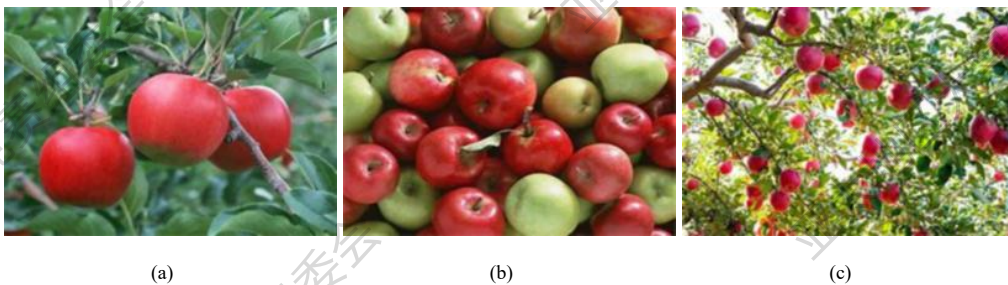




Figure 15 Examples of selected sample images

We randomly selected 100 images as training samples. Here, we provide a few to illustrate that the chosen apple images exhibit a variety of distinctive features.

Then we utilized the Image Labeler toolbox in MATLAB to manually annotate the selected samples by bounding boxes, where each box represents the region containing an apple. After processing, we obtained a 90x2 table. The first column holds the paths of the images, and the second column stores the coordinates of all the bounding boxes in each image.

The results after adding bounding boxes are as follows. Similarly, we provide examples of these images with added bounding boxes:



Figure 16 Results with bounding boxes added

After completing the annotation, all box coordinates are labeled and recorded for subsequent calculation of the position of each apple.

Then, we take the selected 100 processed sample images and randomly divide them into three sets in an 8 : 1 : 1 ratio using the `rand()` function. These sets are referred to as the training set K1, validation set K2, and test set K3, respectively.

Up to this point, we have completed the preprocessing of the training samples.

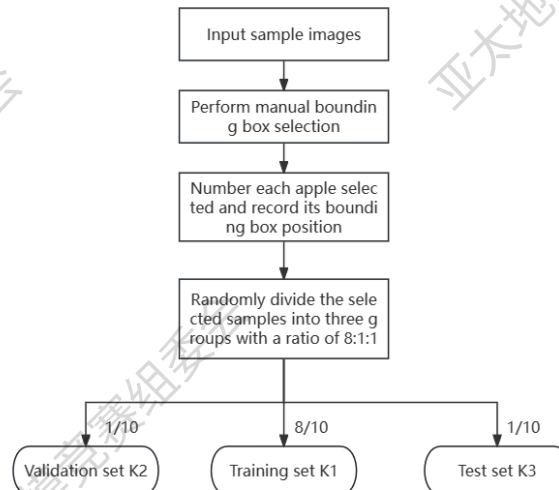


Figure 17 Data preprocessing flowchart of Problem 2

Then we input the samples into the YOLO v4 model for training:

1. First, we perform convolution operation: Perform convolution operation on the training set to extract the feature values of the input information. It's important to note that in the process of scene recognition, we are not instantly recognizing all targets in the entire scene. Instead, we focus on locally perceiving a single feature in the scene and then perform local operations through higher levels to obtain global information. The following diagram shows the structure of the convolutional layer.

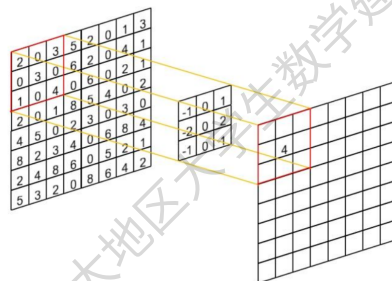


Figure 18 Convolutional Layer Structure Diagram (Image Sourced from the Internet)

2. Next, we enter the pooling layer. The purpose of the pooling layer is to compress the image. It exists between multiple convolutional layers and performs feature dimension reduction on image information. It compresses global information of the image and reduces the number of parameters to prevent overfitting and improve fault tolerance. Pooling can be divided into max pooling and average pooling operations. The following diagram illustrates these two pooling operations.

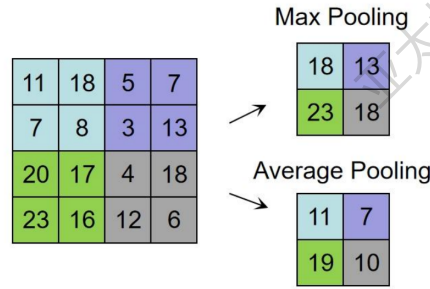


Figure 19 Pooling Layer Structure Diagram (Image Sourced from the Internet)

- Finally, we enter the fully connected layer. Here, we perform fully connected operations on all feature images. If overfitting occurs, we use operations such as dropout and local normalization to randomly delete some neurons generated during the neural network training process.
- After extracting features from the training set, let's denote the extracted feature information as ε . Then, we using ε to recognize the validation set. The number of recognized apples is denoted as m , and their positions are (x_i, y_i) . Subsequently, based on the previously manually selected results, we calculate the number of apples M and their positions (x_{0i}, y_{0i}) in the validation set. We perform confidence calculation for the model and record it in Training Loss.
- Check whether the value of Training Loss satisfies the confidence interval I . If it does not satisfy, then perform convolution operation again on the training set to re-extract feature information, denoted as ε^* . If it satisfies, proceed to the next step.
- Use the feature information that passed the test to examine the test set. Similarly, calculate its feasibility and record it in Validation Loss for subsequent evaluation of the model's credibility.

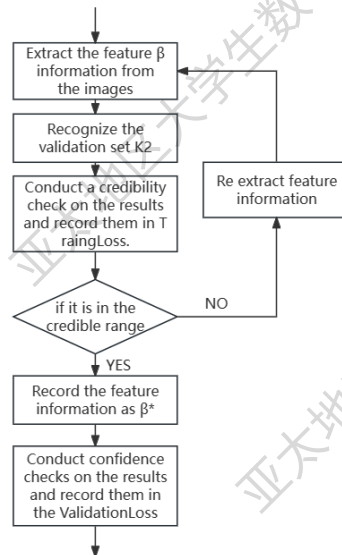


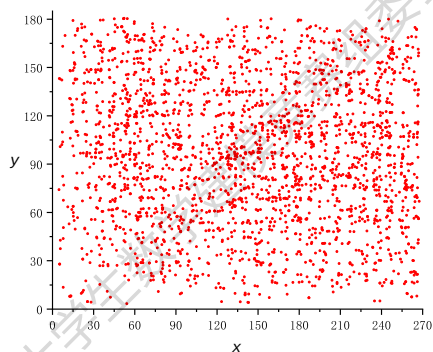
Figure 20 Flowchart of model training(part)

At this point, we have completed the training of the sample model under the YOLO v4 network, which can be used for the subsequent detection of apple positions.

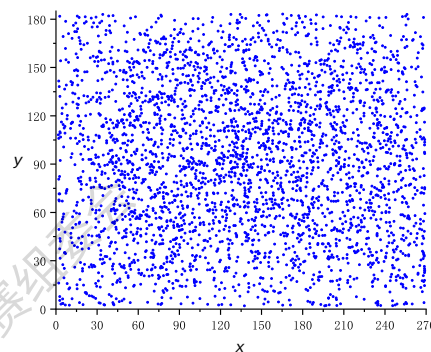
6.1.3 Comparison of Model Recognition and Sensitivity Analysis

We plot the result (the left) against the apple position obtained by Hall circle transformation from question 1 (the right)

After comparison, we found that although there was a slight error in the number of identifications between the two, the distribution of point coordinates generally coincided, which shows that the apple position coordinates we obtained are relatively accurate.



(a) The scatterplot of the results from the model in question 2



(b) The scatterplot of the results from the model in question 2

6.2 Results and 2D Scatter Plot of Geometric Coordinates

Our final scatterplot is as follows: Each point represents the geometric coordinate position of the geometric center of an apple in the figure.

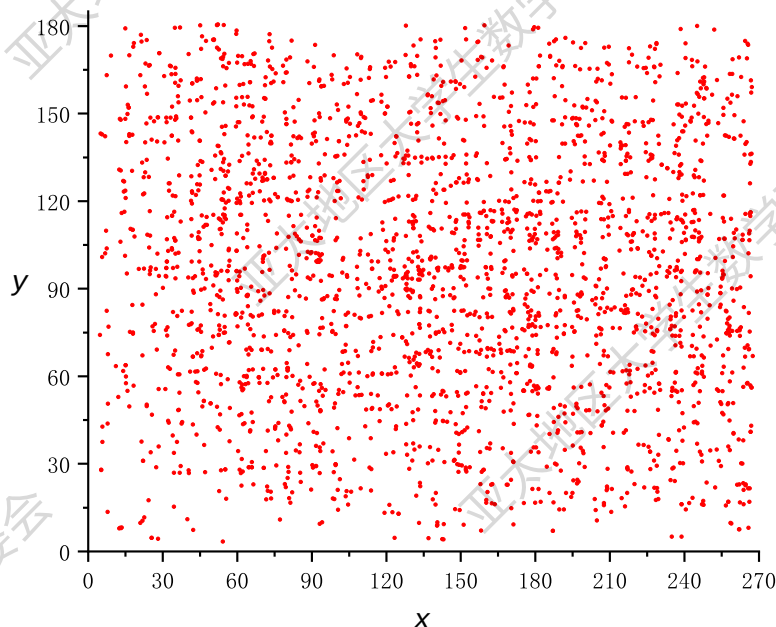


Figure 21 2D Scatter Plot of Geometric Coordinates

VII. Model Establishment and Solution of Problem 3

7.1 Establishment of apple maturity detection model

7.1.1 The Application of YOLOv4 Network and Results in this Problem

Similar to the second question, we randomly selected 100 images as training samples. In addition to the frame addition operation, we also performed a maturity status judgment on each apple, using R, S, and I to represent mature and semi-ripe apples respectively. mature and immature.

We can refer to the model in question 2 for the process of feature information extraction, and will not go into details here. The difference from question 2 is that this time the number of feature information sets we extracted increased. The feature information of mature apples, semi-ripe apples and immature apples is t1, t2, t3.

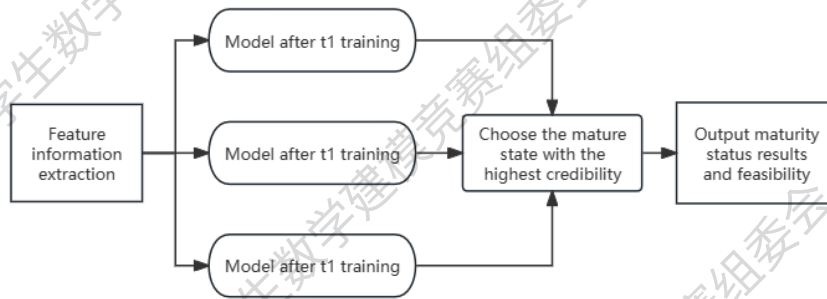


Figure 22 Schematic flow diagram of this process

After the above operations, three types of feature information are used to identify the image of attachment 1 respectively, and the credibility of the apple status is tested by comparing the three types of feature information, and the one with the highest credibility is selected as the judgment for identifying the ripe status of the apple, and It is represented in the figure as follows:
(For example, R: 94% indicates that this apple is mature, and the credibility is 94%)

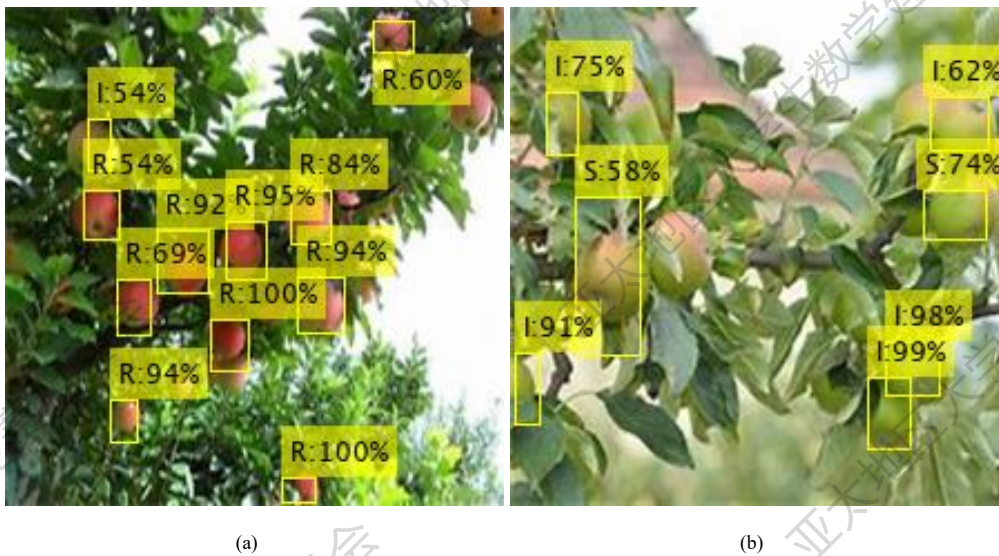


Figure 23 Schematic diagram of maturity identification results

Through the above training process of the model, we can get a rough judgment of the ripeness status of each apple and its credibility. Next, we hope to quantify this information and express it with a unified parameter maturity.

7.1.2 Establishment of numerical calculation model for maturity

Through the data solved in the previous section, we can get the following parameters:

The numbers of ripe, semi-ripe, and immature apples in all images is recorded as A, B, and C.

The ripeness status of each apple $label \in \{R, S, I\}$

The confidence level of each apple's ripeness p

To quantitatively express the maturity of each apple, we introduce the maturity factor μ

$$\mu = \begin{cases} 1, & \text{if label is } R \\ \frac{1}{2}, & \text{if label is } S \\ 0, & \text{if label is } I \end{cases}$$

And we assign the above value to μ according to its ripeness status for each apple.

So we can preliminarily think that the apple's maturity H is related to the product of μ and its credibility p. We record maturity as H

So we can initially get:

$$H = \mu p$$

After testing, we found that the error in calculating maturity in this way is relatively large. Analyzing the reason, we believe that the ripe state of apples is not binary opposition, so it is necessary to consider the possible ripe state of apples under the probability of (1-p). Here we introduce the average maturity $\tilde{H}$, and is given by:

$$\tilde{H} = \begin{cases} \frac{\frac{1}{2} \sum p_{bi}}{B + C}, & \text{if label is } R \\ \frac{\sum p_{ai}}{A + C}, & \text{if label is } S \\ \frac{\sum (p_{ai} + \frac{1}{2} p_{bi})}{A + B}, & \text{if label is } I \end{cases} \quad (6)$$

Among them, p_{ai} , p_{bi} , and p_{ci} respectively represent the credibility of an apple with labels R, S, and I.

So we can get the calculation formula of the modified maturity H:

$$H = \mu p + (1 - p) \tilde{H} \quad (7)$$

Based on this, we can calculate the maturity H of each apple.

7.2 Results and Histogram of apple ripeness distribution

We drew a histogram of apple maturity distribution with every 5% of maturity as interval step. We can find that most of the identified apples are relatively mature, which is consistent with the actual situation. But at the same time, because there are too few immature apples, if they are all

represented in one histogram, the data visualization effect will be poor, so we divide the histogram into two parts and draw them separately, as follows:

Figure 24 Histogram of ripeness distribution of unripe apples

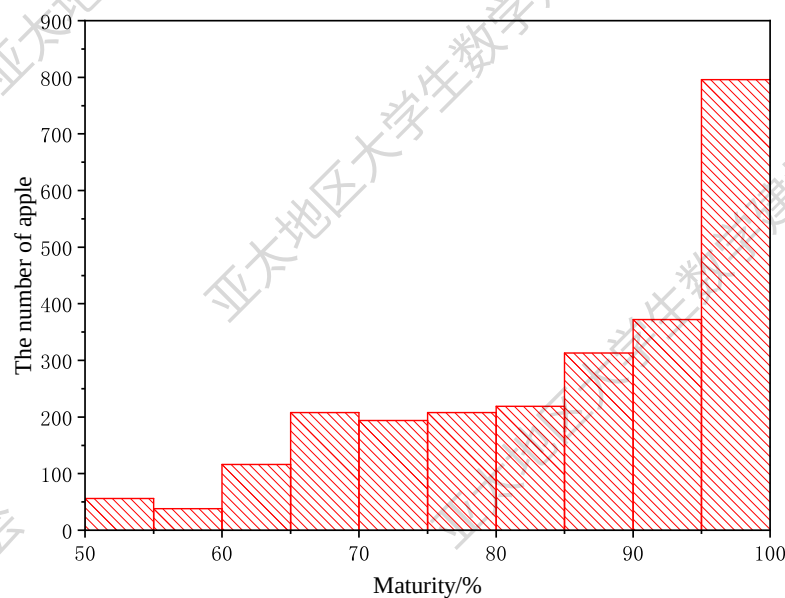
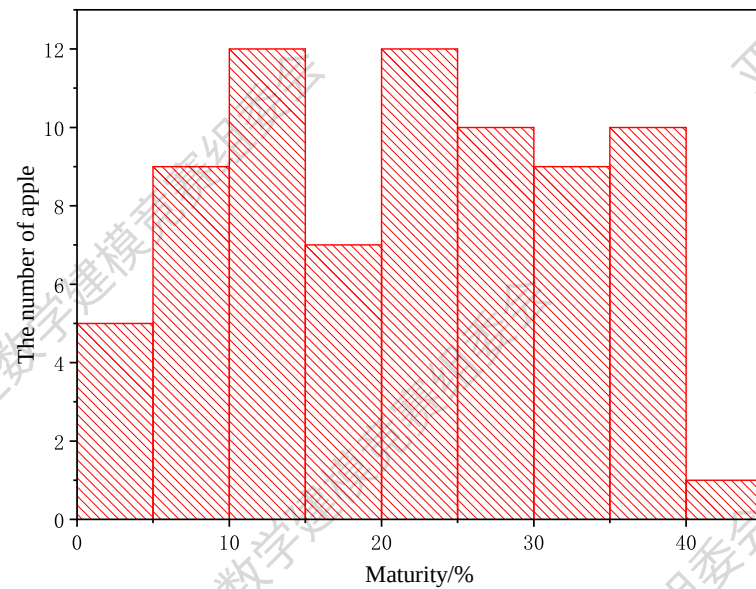


Figure 25 Ripeness distribution histogram of ripe and semi-ripe apples

VIII. Model Establishment and Solution of Problem 4

8.1 Preliminary estimates for Apple

Generally, apples are represented by 70, 80, 90, etc. The unit is mm, indicating the diameter. Here we might as well take the diameter of the apple as $9cm$. According to the image radius rad calculated in question 1, we can approximate Calculate the volume of apple, so we have

$$V = \frac{4}{3}\pi r^3$$

Here we need to determine the focal length of the camera and the distance from which we are shooting. Our focal length is selected as the standard focal length $60mm$. As a preliminary estimate, we do not consider the different positions of the apple but default to the same position of the apple, recorded as D , then we have

$$V_{true} = \frac{\eta^3 D^3}{f^3} \frac{4\pi}{3} r^3$$

Therefore, the mass of the apple is estimated to be

$$m = \rho V_{true} = \frac{4\eta^3 D^3}{3f^3} \pi \rho r^3$$

Next, we need to estimate a reasonable ratio $\frac{D}{f}$ to help us improve accuracy. In fact, the scale of a pixel on a two-dimensional image corresponds to $0.04cm$ in reality (we record this ratio as η), then we can use the area of the two-dimensional image as a rough distance estimate, we A two-dimensional image with a size of 270×185 was segmented, and the result is shown in the figure below

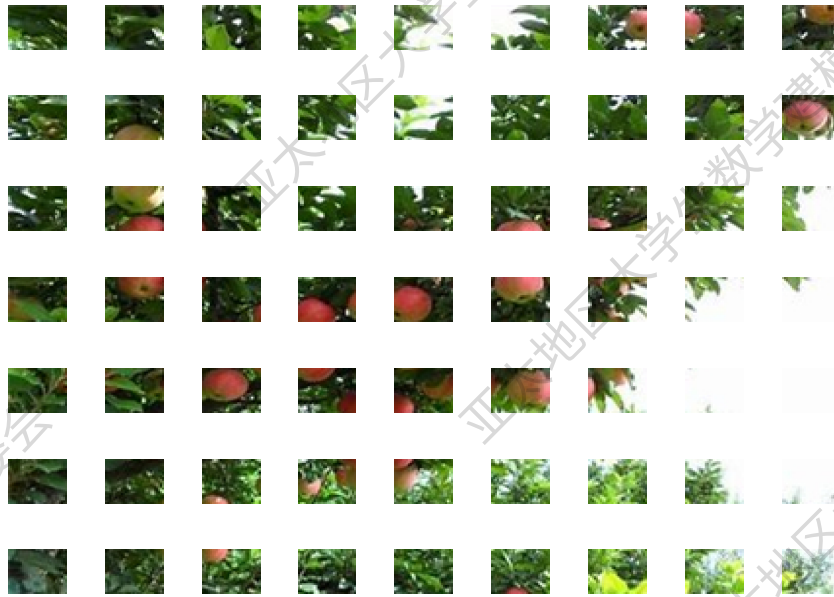


Figure 26 The segmented image

In this segmented picture of 7×9 that is all apples, it is not difficult to estimate that one apple occupies about the size of one and a half squares, so its corresponding image diameter can be estimated to be 1.8cm and the actual diameter is 9cm , so

$$\frac{D}{f} = \frac{9}{1.8} = 5$$

Therefore, the mass of apples is estimated to be

$$m = \frac{4}{375} \pi \rho r^3$$

Using the above formula, we obtain the quality histogram of apple quality, as shown in the Fig.27

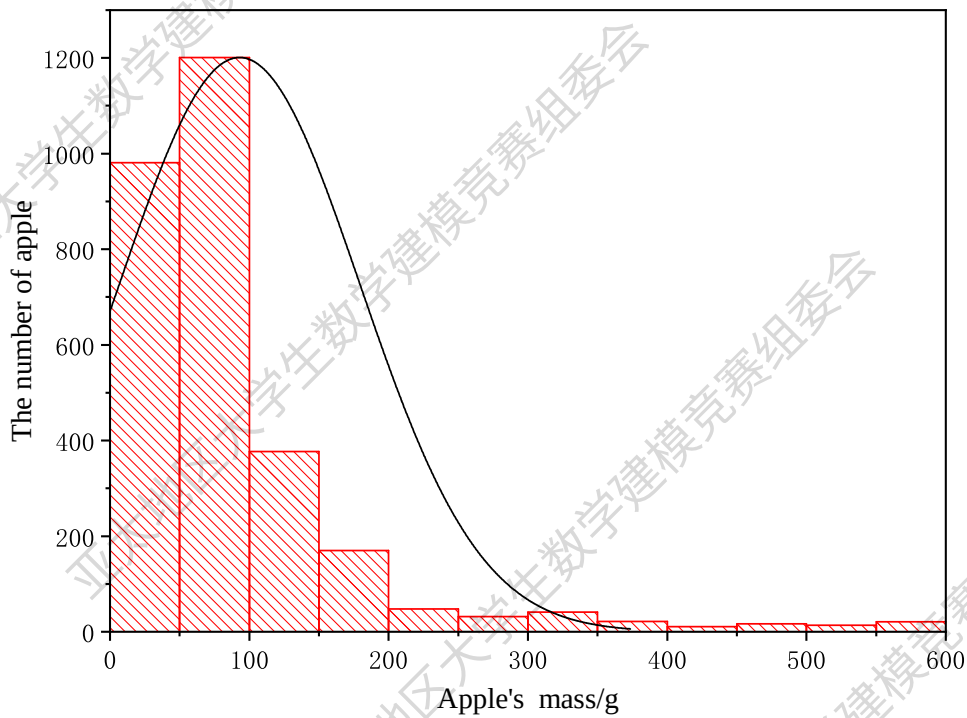


Figure 27 Mass distribution histogram of apples

As can be seen from the image, this estimation result is very inaccurate, so we need to improve our estimation model.

8.2 Radance and radiation intensity

As we all know, radiance L_e is a physical quantity used to characterize the radiant flux per unit area and unit solid angle of a surface radiation source. In addition, in sensors (cameras, human eyes, etc.), radiance is the most commonly used to describe the perceived radiation. Quantity is also the most important quantity.

$$L_e = \frac{d^2 \Phi_e}{\cos \theta dA d\Omega}$$

In the formula, e is the specified direction, Φ_e is the radiation flux in the specified direction, dA is the area element, $\cos \theta$ is the normal angle of the area element, Ω is the solid angle.

Now, let's consider the situation where a digital camera takes a picture. Let's assume that the area element at the distance r is dA , the radiance is L , the light-receiving area (that is, the area of the digital camera) is S , the focal length of the camera is f , and the imaging The generated irradiance is E_{img} . Then, the solid angle formed on the camera surface element can be written as

$$d\Omega = \frac{dA}{r^2}$$

The radiation flux passing through the camera is

$$d\Phi = d\Omega LS = \frac{LS}{r^2} dA$$

Let the image size of surface micro-elements on the camera be dA' , then we have

$$dA' = \frac{f^2}{r^2} dA$$

Therefore, the irradiance imaged by the surface elements is

$$E_{img} = \frac{d\Phi}{dA'} = \frac{LSdA}{r^2} \cdot \frac{r^2}{f^2 dA} = \frac{LS}{f^2} \quad (8)$$

The formula (8) indicates that the imaging irradiance E_{img} of the object will not change with the change of distance d (here we default to the aperture setting, exposure time and other parameters of the camera every time a photo is taken settings are consistent).

8.3 Establishment and calculation optimization of ranging model

The color space we selected previously is the Lab color space, and Lab happens to use numerical values to fit the color space of human visual perception, where the L component corresponds to the brightness perceived by the human eye. Based on this, our team proposed a method to estimate the distance of objects in two-dimensional images by using the L component in the Lab color space.

The basic steps are as follows:

- converts the L component map into a grayscale image, and its value can approximately represent the brightness of the pixel.
- frames the position of the apple in the grayscale image
- accumulates the grayscale values of the pixels in the frame and averages the area. This average value can be regarded as the average brightness of the object H , which is proportional to the radiation intensity of the object E . You may wish to remember the ratio The coefficient is l
- The radiation intensity satisfies the inverse square law with the same distance, and the actual distance of the object can be estimated.

Denote the gray value matrix of each pixel in the two-dimensional grayscale image as $G_{m \times n}$, where $G(i, j)$ represents the i th row and j th column of the grayscale image. For the gray value of the pixel, record the frame range of the i ($i = 1, 2, \dots, k$) th apple on the image as the restriction matrix $X_{k \times 2}, Y_{k \times 2}$ ($X_{k1} < X_{k2}, Y_{k1} < Y_{k2}$). Therefore, the average brightness corresponding to the i th apple on the grayscale image can be expressed as

$$W_i = \frac{\sum_x \sum_y G(x, y)}{\eta^2 (x_{i1} - x_{i2})(y_{i1} - y_{i2})}, \quad i = 1, 2, \dots, k \quad (9)$$

s.t:

$$x \in [X_{i1}, X_{i2}], \quad y \in [Y_{i1}, Y_{i2}]$$

Then according to the definition of radiation intensity, we have

$$lW_i = E_i = \frac{d\Phi_i}{dA_i} = \frac{LS}{\eta^2 r^2}$$

that is

$$r_i = \frac{1}{\eta} \sqrt{\frac{LS}{lW_i}}, \quad i = 1, 2, \dots, k$$

The radiance L in the formula is a physical quantity that does not change with distance, so the distance is a function of the average brightness. The lower the average brightness, the farther the distance, which is in line with our intuitive judgment of the image.

Since we cannot obtain the values of parameters L, S, l , we hope to select a standard image (required to contain only one apple and the identified apple position is highly consistent with the position of the image), and the distance between the apples in the image Denoted as $r_0 = 1m$, the two-dimensional image area is S_0 , and the real area is A_0 , so we have

$$r_i = \frac{r_i}{r_0} = \sqrt{\frac{LS}{lW_i} \cdot \frac{lW_0}{LS}} = \sqrt{\frac{W_0}{W_i}} \quad (10)$$

This is the final result of the ranging model. We eliminate the parameters that we cannot obtain through the ratio, and finally obtain the relationship between ranging and the average brightness of the two-dimensional plane image, which will help us complete the ranging task.

In order to reduce the complexity of the algorithm and optimize the model, we also need to optimize the calculation formula of average brightness. To this end, we introduce the Hadamard product operation of the matrix and record "*" as the Hadamard product operator. In addition, we record the norm $\|A\| = \sum A$ to represent the calculation of the elements of the matrix A and.

Now let's improve the summation operation of the gray value matrix. We set the "0-1" column vectors in the x, y direction as α, β . Their definitions are:

$$\alpha_i(t) = \begin{cases} 1, & \text{if } x_{i1} \leq t \leq x_{i2} \\ 0, & \text{others} \end{cases} \quad t = 1, 2, \dots, m$$

$$\beta_i(t) = \begin{cases} 1, & \text{if } y_{i1} \leq t \leq y_{i2} \\ 0, & \text{others} \end{cases} \quad t = 1, 2, \dots, n$$

Among them, $\alpha_i(t), \beta_i(t)$ respectively represent the value of the t th row of the "0-1" column vector of the i th apple. We might as well remember the range matrix as

$$R_i = \alpha_i \beta_i^T$$

It is worth pointing out that the range matrix R_i generated by α_i, β_i , itself is also a "0-1" matrix containing only 0, 1 elements. Therefore, the summation operation of the gray value matrix can be rewritten as a matrix Hadamard product operation, that is

$$\sum_x \sum_y G(x, y) = \|G * R\| \quad (11)$$

It is not difficult to find that x, y here no longer need to limit its value range but can traverse the entire gray value matrix. We do not have to worry that this will cause the algorithm to slow down, because this is essentially a basic matrix operation.

As an example, we might as well consider the gray value matrix $G_{4 \times 4}$ of a two-dimensional grayscale image of 4×4 as

$$G = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

The frame range of an apple above it is $[1, 2] \times [2, 3]$, then

$$\alpha = \begin{bmatrix} 1 & 1 & 0 & 0 \end{bmatrix}^T$$

$$\beta = \begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix}^T$$

Then the range matrix is

$$R = \alpha\beta^T = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

So the result is

$$\|G * R\| = \left\| \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \right\| = \left\| \begin{bmatrix} 0 & 2 & 3 & 0 \\ 0 & 6 & 7 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \right\| = 18$$

Next, we optimize the area calculation for the framed range. Note that the area S of the framed range can be expressed as

$$S_i = \eta^2(x_{i1} - x_{i2})(y_{i1} - y_{i2}) = \eta^2 \|\alpha_i \beta_i^T\|$$

Therefore the area column vector is

$$S = \begin{bmatrix} S_1 \\ S_2 \\ \vdots \\ S_k \end{bmatrix} = \eta^2 \begin{bmatrix} \|\alpha_1 \beta_1^T\| \\ \|\alpha_2 \beta_2^T\| \\ \vdots \\ \|\alpha_k \beta_k^T\| \end{bmatrix}$$

Then

$$r_i = \sqrt{\frac{W_0}{W_i}}, \quad i = 1, 2, \dots, k$$

In

$$W_i = \frac{\|G * R_i\|}{S_i} = \frac{\|G * R_i\|}{\eta^2 \|\alpha_i \beta_i^T\|}, \quad i = 1, 2, \dots, k$$

That is

$$r = \eta \sqrt{\frac{\|\alpha \beta^T\|}{\|G * R\|} W_0} \quad (12)$$

This is the optimized relative distance calculation formula, which can be used to approximately represent the real distance. After successfully solving the problem of different fruit distances, we will next complete the calculation of the actual area and mass of the fruit.

8.4 Estimate of actual fruit size and mass

Suppose the focal length of the camera is f , the distance of the fruit is r_{true} , and the area of the two-dimensional plane is S . According to the imaging and area similarity laws, the relative actual size A_{rel} can be obtained for

$$A' = \frac{r_{true}^2}{f^2} S$$

The reason why the above formula is a relative area is that the above formula is equivalent to

$$\begin{aligned} A' &= \frac{r_0^2 r_0^{-2}}{f^2 r_0^{-2}} S \\ &= \frac{r_{rel}^2 A_0}{S_0} S \end{aligned}$$

That is

$$A_{rel} = \frac{A'}{A_0} = r_{rel}^2 \frac{S}{S_0}$$

Substituting the previously obtained formula into, we have

$$A_{rel} = \frac{W_0 \eta^2}{f^2} \frac{\|\alpha \beta^T\| S}{\|G * R\| S_0} = \frac{1}{f^2} \frac{\|\alpha \beta^T\|}{\|\alpha_0 \beta_0^T\|} \frac{W_0}{W_i}$$

This is the area enclosed by the box. Considering that the box is basically tangent to the edge of the apple, and the apple can be approximately regarded as a circle, we need to measure the relationship between the area of the box and the area of the fruit. To this end, we introduce The new parameter δ helps us accomplish this task, which is defined as

$$\delta = \frac{\pi r^2}{S} \approx \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

That is

$$A = \delta A_{rel} = \frac{\pi}{4f^2} \frac{\|\alpha\beta^T\|}{\|\alpha_0\beta_0^T\|} \frac{W_0}{W_i}$$

Then, for the mass calculation of apples we can make the following estimate

$$\begin{aligned} m &= \frac{4\pi\rho}{3} \sqrt{\left(\frac{A}{\pi}\right)^3} \\ &= \frac{\pi\rho}{6f^3} \left(\sqrt{\frac{\|\alpha\beta^T\|W_0}{\|\alpha_0\beta_0^T\|W_i}} \right)^3 \end{aligned}$$

In this way, the estimate of quality can be calculated entirely from the brightness values on the two-dimensional plane.

8.5 Results and mass distribution histogram

Figure 28 Mass distribution histogram of apples

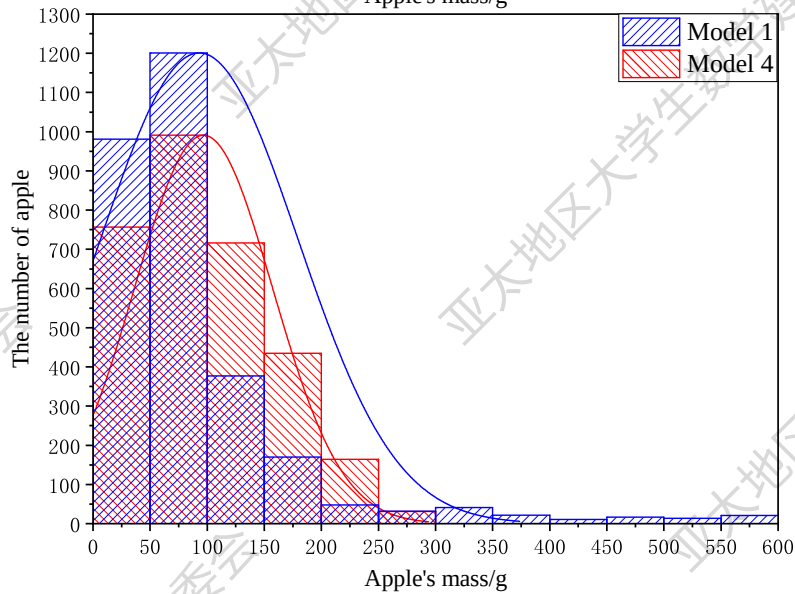
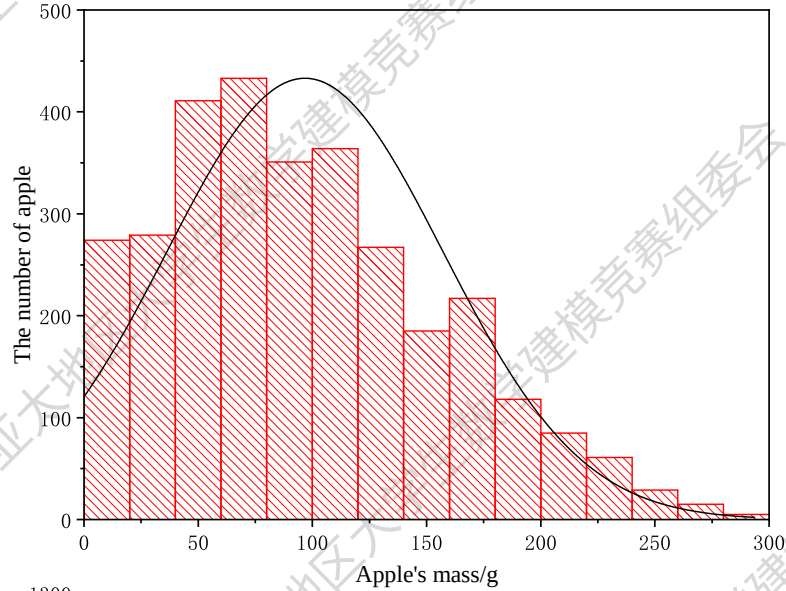


Figure 29 Mass distribution histogram of apples

From the comparison chart between Model 1 and Model 4, we can know that the model results we established are quite good, which also confirms the rationality of our modeling.

IX. Model Establishment and Solution of Problem 5

After consulting the relevant literature, we can learn that YOLOv4 is often used for target detection, while VGG19 is often used for image classification. Both belong to convolutional neural networks (CNN).

So we choose VGG19 for solving Problem 5.

9.1 Establishment of an improved apple recognition model

9.1.1 Principle Overview of VGG19 network

One of the most famous object recognition networks at the time was called VGG19. Each layer of the neural network would use the output of the previous layer to further extract more complex features until it was complex enough to be used to identify objects, so each layer Each layer can be regarded as an extractor of many local features.

The following is the principle description of VGG19:

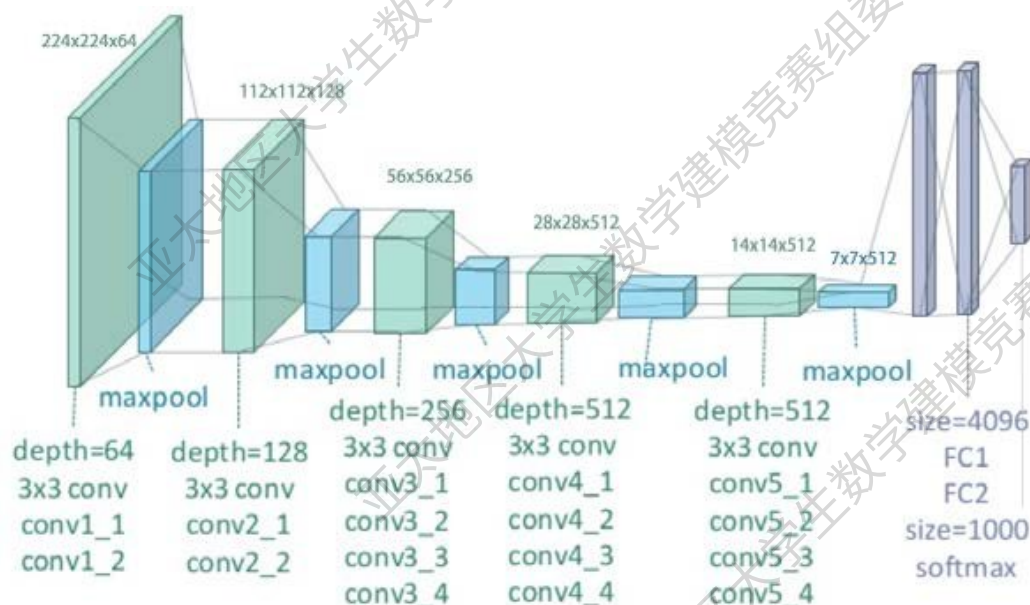


Figure 30 Partial structural diagram of VGG19(Image Sourced from the internet)

VGG19 network model, which includes 5 convolution modules, 5 pooling layers, 3 fully connected layers and 1 softmax classifier. Among them, 5 convolution modules contain a total of 16 convolution layers. The convolution kernel size of the VGG19 network model is 3×3, and the step size is 1.

The core operation of the convolutional neural network is the convolution operation, which is shown in Figure 2. In Figure 2, a 3×3 convolution kernel slides on the original image, and the covered value is multiplied by the corresponding position of the weight on the convolution kernel

and then added. The obtained value is filled in the corresponding position of the feature map, that is, extracted The features of the original image are reflected on the feature map.

Each convolution kernel extracts one feature and generates one feature map. Then, by stacking 64 feature maps of 64 convolution kernels, a three-dimensional feature map will be obtained. Convolution kernels in different convolution layers are used to extract different feature information. Shallow convolution kernels mainly extract pixel features such as color, edges and patches; middle-level convolution kernels mainly extract texture features such as stripes and shapes; high-level convolution kernels Extract more abstract semantic features such as text.

In order to compare the quality of image stitching, this article selects the features extracted from the 1st to 5th convolutional layers in the VGG19 network model as the main comparison features.

After each layer of convolution operation is completed, a three-dimensional feature map stacked by two-dimensional feature maps will be obtained. The three-dimensional feature map is shown followed.

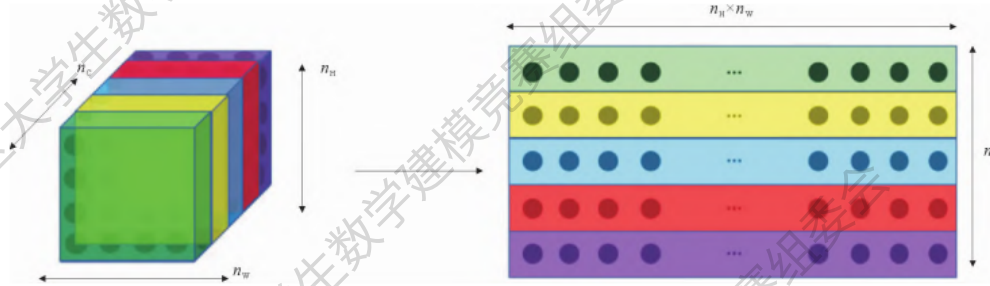


Figure 31 Three-dimensional feature map transformation (Image Sourced from the internet)

Each color of the feature map represents a two-dimensional feature map obtained by convolution with a convolution kernel. This article takes the fifth layer as an example for analysis. 256 convolutions After the kernel completes the convolution operation, a $56 \times 56 \times 256$ feature map is obtained. In the figure, $n_w = 56$, $n_h = 56$, $n_c = 256$. Analyze the second layer of the first layer separately. dimensional feature map, which can be expressed as a 56×56 matrix F_1

$$F_1 = \begin{bmatrix} a_1 & a_2 & \dots & a_{56} \\ a_{57} & a_{58} & \dots & a_{112} \\ & & \vdots & \\ a_{3081} & a_{3082} & \dots & a_{3136} \end{bmatrix}$$

Similarly, define the matrix of the second layer feature map as F_2 , and so on. Each feature map corresponds to a matrix, and the last feature map corresponds to a matrix. F_{256} . Then each matrix is flattened into a row vector, that is, the matrix F_1 is transformed into

$$H_1 = [a_1 \ a_2 \ \dots \ a_{3136}]$$

Then the three-dimensional feature map of the convolutional layer will be transformed accordingly.

The entire feature map can be expressed as a 3136×256 matrix K , that is

$$K = \begin{bmatrix} H_1 & H_2 & \dots & H_{255} & H_{256} \end{bmatrix}^T$$

At this time, the Gram matrix can be expressed as

$$G = K K^T$$

In the formula, K^T is the transposed matrix of matrix K . Obviously the matrix G is a 256×256 symmetric matrix, which represents the 256 feature maps of the fifth convolution layer. The co-occurrence correlation between the 256 features in the table can be used to represent the style of an image, and then the style of the two images can be represented by the co-occurrence correlation. The degree of difference determines the quality of image splicing. The function L_s is used as an index to compare the degree of style difference between two images, that is

$$\begin{cases} L = \sum_{k=0}^m w_k E_k \\ E_k = \frac{1}{4N_k^2 M_k^2} \sum_{i,j} (G_{i,j}^k - A_{i,j}^k)^2 \end{cases}$$

G^k is the Gram matrix of the original image in the k convolution layer; A^k is the Gram matrix of the spliced image in the k convolution layer; N^k is the 1st layer of the two images. The number of channels of the feature map; M^k is the product of the length and width of the k -th layer feature map of the two images; w_k is the weight of the k -layer image.

9.1.2 The Application of the VGG19 network

We apply the principle of this network, take all the apple images provided in Appendix 2 as samples, use the VGG19 network for deep learning, and use the trained model for detection in Appendix 3.

Here we use MATLAB for simulation, and the recognition results are as follows:



(a) The recognition result is tomato

(b) The recognition result is apple

Figure 32 Apple recognition result window after VGG19 training

9.2 Results and histogram for all Apple image ID numbers

The results of the model we trained through VCG19 for recognition are as follows: The abscissa is the ID of the image. Starting from the first one, we take every 1,000 pictures as a grid. The ordinate is the number of images identified as apples in these 1,000 pictures. The result should be between 0 and 1,000. After comparison, it is found that all All comply.

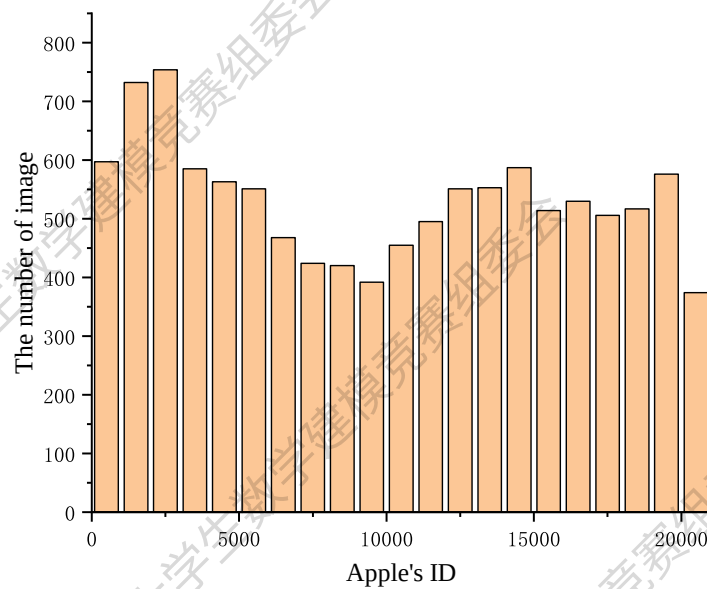
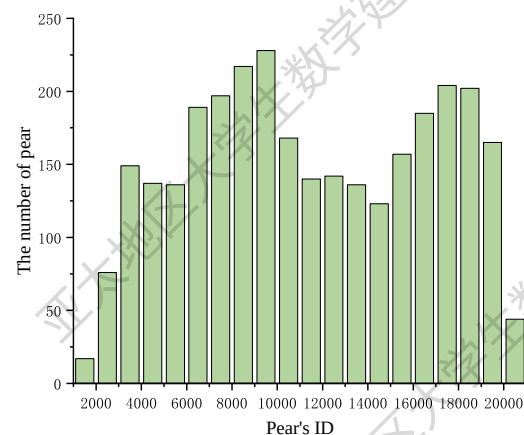
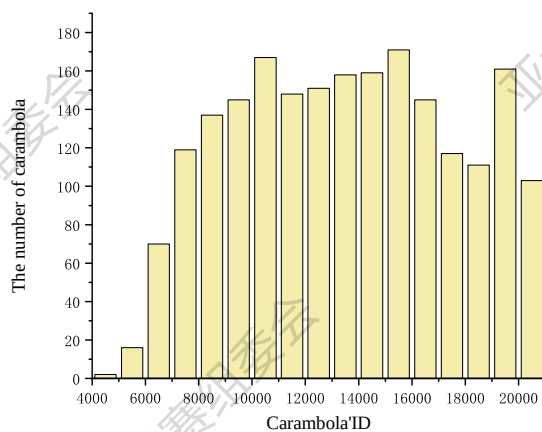


Figure 33 Histogram for all Apple image ID numbers

According to the model we trained, not only apples can be recognized, we also recognized other fruits and drew the distribution histogram of their image ID numbers. The results are as follows:



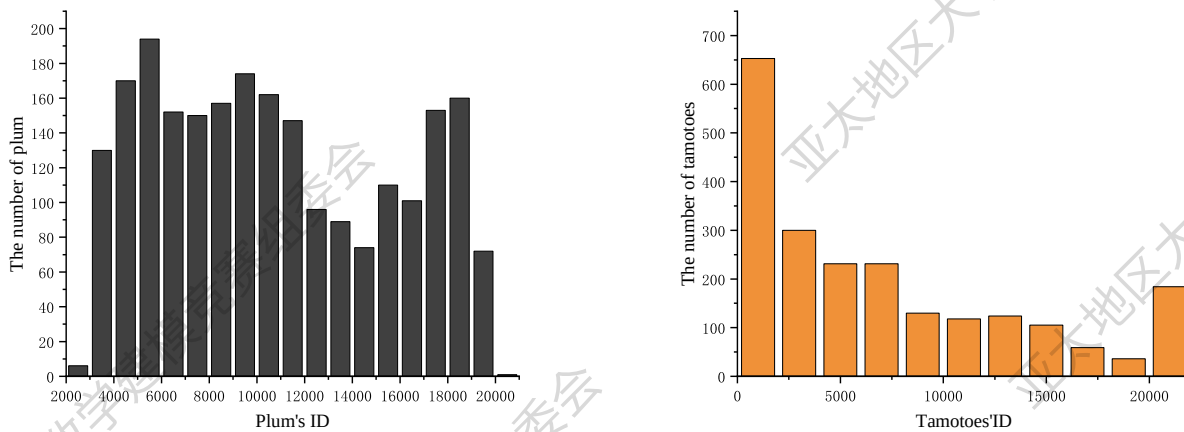


Figure 34 Histograms for other fruits' image ID numbers

X. Model Evaluation and Further Discussion

10.1 Merit

- **Model for Problem 1**'s advantages are that the algorithm is fast and time-consuming, it can accurately recognize images with obvious apple features and the drawn circular boundary is highly consistent with the edge of the apple;

- Compared with the deep learning algorithm, the algorithm does not require pre-training. It only needs to adjust the parameters to directly detect and identify apple images. This is more suitable for scenes that require temporary detection;

- The output of the algorithm also includes the center coordinates and radius parameters of the detected circle, which facilitates our team to establish a coordinate system and solve the following problems.

- **Model for Problem 2 and 3** Using the YOLOv4 network, the training speed is fast and the model memory footprint is small

- **Model for Problem 4** is derived through strict mathematical means. From a theoretical point of view, the model is scientific and reliable enough. In addition, we also made full use of the results of the problem 1 model to test the model. The result is that the speed and accuracy of the model have been improved compared with model 1;

- In this model, we fully considered the large and small situations in the imaging image, and innovatively proposed a method of estimating image distance using image brightness - the brightness estimation method. The final results show that this method is very effective. Reasonable and effective.

- **Model for Problem 5** Using the VGG19 network, the model has a high training depth and a large number of parameters. After training, it can achieve high accuracy.

10.2 Shortcoming

- **Model for Problem 1** relies heavily on the color features and shape features of apples and is more sensitive to the recognition of color features. This will cause some things with similar color features but not apples to be mistakenly detected as apples. At the same time, the recognition effect of green apples is poor;

- The parameter selection of this model is a matter of high trial and error cost, and the adjustment of parameters is even more laborious. Our team spent a lot of time adjusting appropriate parameters to make the detection results as possible as possible. Good.

- **Model for Problem 2 and 3** The training depth of YOLOv4 network is not enough, and misjudgments may occur.

- **Model for Problem 4** The derivation of this model contains some approximate estimates, which will reduce the accuracy of the model's calculation results;

- The establishment of the model requires many conditions. For example, it is assumed that the imaging is all natural light and the influence of other light sources is excluded. At the same time, the accuracy of the model is also affected by the accuracy of the previous Apple detection model.

- **Model for Problem 5** The VGG19 network model is relatively complex, takes a long time to train, is difficult to adjust parameters, has large storage capacity, and has high requirements for GPU performance.

10.3 Improvement Measures

- **Model for Problem 1** But in fact, we can also enhance the sensitivity of the algorithm to the apple shape characteristics, that is, the accuracy of the test, by correcting the detection results of the center of the circle multiple times. We used the gradient method in this article, so we can correct the circular detection results and reduce the false detection rate by drawing a circle with three points.

- **Model for Problem 4** In the model, we assume parameters such as focal length to help our team build the model, which is also the core of the entire model. In real life, when we take an image, we can record the focal length of the image at the same time, so we can perform separate calculations for each image, which makes the results more accurate. In addition, we can also keep the shooting distance as consistent as possible and the lighting conditions equal as much as the error allows. Under such actual conditions, the accuracy of the model will be greatly improved.

XI. References

- [1] Li Changlu. Research on identification and positioning of apple picking robots based on machine vision [D]. Lanzhou University of Technology, 2023. DOI: 10.27206/d.cnki.gsgsu.2022.000718.
- [2] Zhang Junkai, Li Xin, Han Junxian, etc. Research on optimization of picking robot system under deep learning algorithm [J]. Agricultural Mechanization Research, 2024, 46(04): 58-62. DOI: 10.13427/j.cnki.njyi.20230728.001.
- [3] Yang Chunyan. A circle detection method based on Hough transform [J]. Journal of Baicheng Normal University, 2016, 30(02): 65-68.
- [4] Wang Yong, Tao Zhaosheng, Shi Xinyu, etc. Apple target detection method with different maturity levels based on improved YOLOv5s [J/OL]. Journal of Nanjing Agricultural University: 1-13[2023-11-26].<http://kns.cnki.net/kcms/detail/32.1148.S.20230926.1201.002.html>.

- [5] Ma Fangda, Liu Zeping, Chen Shihai, etc. Image stitching quality evaluation algorithm based on VGG19 convolutional neural network [J]. Journal of Qingdao University (Engineering Technology Edition), 2023, 38(02): 31-36. DOI:10.13306/j.1006-9798.2023.02.005.
- [6] Xu Weisen. Research on target recognition algorithm for agricultural picking apples[J]. Electronic World, 2018(14):44-45.DOI:10.19353/j.cnki.dzsj.2018.14.017.
- [7] Liu Xiaoyang. Research on image recognition algorithm of multifunctional fruit and vegetable picking robot[D]. Jiangsu University, 2021.DOI:10.27170/d.cnki.gjsuu.2020.000972.

XII. Appendix

Listing 1: The matlab Source code of Algorithm

```
% The code of Problem 1
clear
clc

% Generate file path, 200 is the number of images in the folder
Fpath="xxx"; % Fill in the file path at xxx
Fspath="xxx"; % Fill in the file path at xxx
for i=1:200
    path(i,1)=Fpath+num2str(i)+".jpg";
    spath(i,1)=Fspath+num2str(i)+".jpg";
end

% Counting matrix, circle center matrix and radius matrix
count=[];
center=[];
rad=[];
for num =1:200
    % Get image data
    data = imread(path(num,1));
    % Convert to lab space
    ldata = rgb2lab(data);
    % only a* component is required
    gdata = ldata(:,:,2);

    % Set the number of filters
    n=4;
    % Start filtering
    for M=1:n
        mean_gdata=mean(gdata(:));
        var_gdata=var(gdata(:));
        for i=1:size(gdata,1)
            for j=1:size(gdata,2)
                if gdata(i,j)<mean_gdata+sqrt(var_gdata)
                    gdata(i,j)=0;
                end
            end
        end
    end

    % Calculate the threshold k of Gaussian filter
    k = graythresh(gdata);
    % Gaussian filter
    W = fspecial('gaussian',[5,5],2/3);
    G = imfilter(gdata, W, 'replicate');
    % OTSU split
    bw=im2bw(G,k);
```

```

% Hough circle transformation
[centers, radii] = imfindcircles(bw,[10 70],"ObjectPolarity","bright",
    "Sensitivity",0.88,"EdgeThreshold",0.10,"Method","twostage");
% draw circle
imshow(data)
% viscircles(centers,radii);
% Generate distance matrix
for i=1:size(centers,1)
    for j=i+1:size(centers,1)
        d(i,j)=sqrt((centers(i,1)-centers(j,1))^2+(centers(i,2)-centers(j,2))^2);
    end
end
% Generate position vector
p=[];
row=1;
% Marker for overlapping circle position
for i=1:size(centers,1)
    for j=i+1:size(centers,1)
        if max(radii(i),radii(j))>d(i,j)+0.4*min(radii(i),radii(j)) && d(i,j)~=Inf
            if radii(i)<radii(j)
                d(i,:)=Inf;
                d(:,i)=Inf;
                p(row,1)=i;
                row=row+1;
            elseif radii(j)<radii(i) && d(i,j)~=Inf
                d(j,:)=Inf;
                d(:,j)=Inf;
                p(row,1)=j;
                row=row+1;
            end
        end
    end
end
% Sort the position vectors
p=sort(p);
% Eliminate overlapping circles
for i=row-1:-1:1
    centers(p(i,1),:)=[];
    radii(p(i,1),:)=[];
end
% generate image
imshow(data)
viscircles(centers,radii)
f=gcf;
exportgraphics(gcf,spath(num,1))
close
% count
count=size(radii,1);
center{num,1}=centers;

```

```

rad{num,1}=radii;
end

% The code of Problem 4

clear
clc
k=1;
rho=0.75;
f=0.6;
load('bright.mat');
load('center.mat')
load('rad.mat')
for i =1:200
centers=round(center{i,1});
for j = 1:size(rad{i,1},1)
alpha=ones(185,1);beta=ones(1,270);
alpha(max(centers(j,1)-rad{i,1}(j,1),1):min(centers(j,1)+rad{i,1}(j,1),185),1)=1;
beta(1,max(centers(j,2)-rad{i,1}(j,1),1):min(centers(j,2)+rad{i,1}(j,1),270))=1;
R=alpha*beta;
lin=bright{i};
H{i,j}=sum(sum(lin.*R))/(4*rad{i,1}(j,1)^2);
m(k,1)=pi*rho/(6*f^3)*(rad{i,1}(j,1)^3)*sqrt(pi^3*H{i,j}^3/827^3);
k=k+1;
end
end

k=1;
for i=1:200
for j=1:size(rad{i,1},1)
stand(k,1)=log(H{i,j}^3/rad{i,1}(j,1)^6);
stand(k,2)=i;
stand(k,3)=j;
k=k+1;
end
end

```

Listing 2: The lingo source code

```

kk=2;
[mdd,ndd]=size(dd);
while ~isempty(V)
[tmpd,j]=min(W(i,V));tmpj=V(j);
for k=2:ndd
[tmp1,jj]=min(dd(1,k)+W(dd(2,k),V));
tmp2=V(jj);tt(k-1,:)= [tmp1,tmp2,jj];
end
tmp=[tmpd,tmpj,j;tt];[tmp3,tmp4]=min(tmp(:,1));
if tmp3==tmpd, ss(1:2,kk)=[i;tmp(tmp4,2)];

```

```
else,tmp5=find(ss(:,tmp4)~=0);tmp6=length(tmp5);
if dd(2,tmp4)==ss(tmp6,tmp4)
    ss(1:tmp6+1,kk)=[ss(tmp5,tmp4);tmp(tmp4,2)];
else, ss(1:3,kk)=[i;dd(2,tmp4);tmp(tmp4,2)];
end;
end
dd=[dd,[tmp3;tmp(tmp4,2)]];V(tmp(tmp4,3))=[];
[mdd,ndd]=size(dd);
kk=kk+1;
end;
S=ss;
D=dd(1,:);
```